PUSL3190 Computing Project

Project Interim Report

Cre8tify Web Application

Supervisor: Miss Dharani Rajasinghe

Name: Mahara Mahara
Plymouth Index Number: 10952728
Degree Program: BSc (Hons) Software Engineering

Table of Contents

Chapter 01: Introduction 5
1.1. Introduction 5
1.2. Problem Definition 5
1.3. Project Objectives 6
1.3.1. Functional Deliverables 6
1.3.2. Quality Criteria and Standards 6
1.3.3. Success Metrics and Acceptance Criteria 7
Chapter 02: System Analysis 8
2.1. Facts Gathering Techniques 8
2.2. Existing System 9
2.3. Drawbacks of the Existing System 9
2.4. User Persona Analysis 9
2.4.1. Persona 1: The Aspiring Designer (The Seller) 9
2.4.2. Persona 2: The Design Enthusiast (The Buyer) 10
2.4.3. Persona 3: The System Administrator (Internal User) 10
Chapter 03: Requirement Specification 11
3.1. Functional Requirements 11
3.1.1. Customer and User Interface Requirements 11
3.1.2. Designer Module Requirements 11
3.1.3. Administrative and Oversight Requirements 11
3.2. Non-functional Requirements 12
3.3. Hardware/ Software Requirements 12
3.3.1. Development Environment 12
3.3.2. Client-Side (User) Requirements 13
3.4. Technical Constraints and Image Specifications 13
Chapter 04: Feasibility Study 14
4.1. Operational Feasibility 14
4.2. Economical Feasibility 14
4.3. Technical Feasibility 15
Chapter 05: System Architecture 16
5.1. Use Case Diagram 16
5.2. Class Diagram 17
5.3. ER Diagram 18
5.4. High-Level Architectural Diagram 20
Chapter 06: Development Tools and Technologies 22

6.1. Development Methodology 22
6.2. Programming Languages and Tools 22
6.3. Third-Party Components and Libraries 22
6.4. Algorithms 23
Chapter 07: Implementation Progress 24
7.1. Development Environment Setup 24
7.2. Implemented Features 24
7.3. Screenshots / Code Snippets 25
7.3.1. Database Schema and Persistence (MongoDB Compass) 25
7.3.2. Security Layer: Authentication Middleware (authMiddleware.js) 26
7.3.3. Data Modeling: User Entity (User.js) 27
7.3.4. RESTful Routing Architecture (designRoutes.js) 28
7.3.5. Business Logic: Design Controller (designController.js) 29
7.3.6. Frontend Integration: User Login and State (Login.tsx) 30
7.3.7. User Interface (UI) and Frontend Presentation 31
7.4. Challenges Encountered and Solutions 36
7.5. Current System Limitations 36
Chapter 08: Discussion and Summary 38
8.1. Summary of the Report 38
8.2. Deviations from the Original Proposal 38
8.3. Discussion of Implementation and Results 39
8.4. Future Plans and Upcoming Work 39
References 40

List of Figures

Figure 1: Use Case Diagram 16
Figure 2: Class Diagram 17
Figure 3: ER Diagram 18
Figure 4: High Level Architectural Diagram 20
Figure 5: MongoDB Compass 25
Figure 6: Security Layer 26
Figure 7: Data Modelling 27
Figure 8: RESTful Routing Architecture 28
Figure 9: Business Logic 29
Figure 10: Frontend Integration 30
Figure 11: Landing Page 31
Figure 12: Designer SignUp 32
Figure 13: Designer Dashbaord 32
Figure 14: Designing Page 33
Figure 15: My Shop 33
Figure 16: Customer Dashboard 34
Figure 17: Profile 34
Figure 18: Women Collection 35
Figure 19: Live Preview 35

List of Tables

Table 1 36

Chapter 01: Introduction

1.1. Introduction

The global retail landscape has undergone a significant paradigm shift, transitioning from mass production to mass personalization. This global shift is evidenced by the custom apparel market's projected growth of over $2.45 billion by 2029, driven by increasing smartphone penetration and consumer desire for unique garments (Technavio, 2025). As a result, digital platforms have become an essential medium for both consumers and creators to connect and collaborate. Mainly this occurred in the fashion industry, where individuals prefer unique designs that reflect their personal style while creative local designers wish to showcase their artworks. With that, **"Cre8tify"** sits as the intersection between these trends, providing a web-based, mobile responsive e-commerce platform specially developed focusing on **Sri Lankan** custom apparel market, which generated over $636 million in 2024 and continues to grow at an annual rate of 15-20% (ECDB, 2024).

1.2. Problem Definition

This project addresses the lack of localized platforms that allows both Sri Lankan designers and buyers to collaborate within a single platform for custom t-shirt designing and selling. In the modern era, many youths and young adults place significant emphasis on their outfits, seeking unique designs that reflect their personal style. As a result, numerous individuals turn to online custom design platforms, which enable them to align their clothing choices with their preferences and customize them accordingly. At the same time, there is a growing group of creative young individuals who are passionate about designing and aspire to market and sell their own products.

But existing platforms like "Printify" doesn't allow free service from start to end. Also, when it comes to building a market place, the local beginners may find it difficult since platforms like "Printify" focus on global market and they rely on international payment methods like PayPal which is not fully supported in Sri Lanka.

Therefore, local designers face difficulties in finding free platforms to design and sell their designs/products. On the otherside, most of the buyers do not get to virtually fit on their clothes before purchasing. Therefore, it creates a gap between the creativity and accessibility due to lack of customization options, local payment methods and interactive previews. As a result, by building a locally adaptable custom t-shirt designing platform, we can help both the young designers and design enthusiastic buyers, bringing Sri Lanka's e-commerce industry into another extent while also awakening the hidden creative minds.

Therefore, the main problem addressed in this project is the lack of localized, user- friendly, and affordable platform where we can design, preview, and buy custom design t-shirts while allowing designers across our country to showcase and sell their designs for an affordable price within Sri Lanka. My experiences as a digital designer have given me insight into some of the major barriers that Sri Lankan creators face when using global platforms like Printify, particularly regarding complex Etsy integrations and restricted PayPal payment domesticity. These challenges inspired me to develop a localized ecosystem, Cre8tify, enabling Sri Lankan

beginners to design, market, and monetize their creative output within a seamless local framework.

1.3. Project Objectives

The overarching goal of this research project is to design, develop, and evaluate a fully functional prototype of the “Cre8tify” web application. This platform serves as a specialized e-commerce ecosystem that democratizes the apparel design process for Sri Lankan creators while offering a high-fidelity visualization experience for consumers.

The specific objectives are categorized into functional deliverables, quality standards, and success metrics as follows:

1.3.1. Functional Deliverables

The system is engineered to provide a comprehensive end-to-end workflow for three distinct user roles. The core functional objectives include:

- **Development of an Interactive Design Toolbox:** To engineer a user-friendly, browser-based design engine that allows beginner designers to create, manipulate, and upload unique graphics without requiring professional-grade software.
- **Implementation of a Dynamic Visualization Engine:** To integrate a "Live Preview" feature using advanced image processing methodologies. This allows users to visualize designs by overlaying them on uploaded personal photos or dummy models. Furthermore, the project aims to implement a 360° visualization module to ensure a comprehensive view of the apparel design before purchase.
- **Role-Based Architectural Dashboards:** To develop secure, individualized dashboards for Customers, Designers, and Administrators. These interfaces are designed to facilitate portfolio management for creators and quality oversight for administrators.
- **Localized Financial Integration:** To develop a secure, simulated payment module (Sandbox/Bank Transfer) tailored specifically to the Sri Lankan banking context, bypassing the limitations of global payment gateways.

1.3.2. Quality Criteria and Standards

To ensure the prototype meets both academic and professional industry standards, the following quality benchmarks have been established:

- **Responsive Cross-Platform Accessibility:** The application must maintain a consistent layout and full functionality across a spectrum of devices, including desktops, laptops, tablets, and mobile screens, ensuring a seamless "mobile-first" experience.

- **Performance Optimization:** A critical objective is to ensure that high-resolution design previews and complex visualization models load within a threshold of 3–5 seconds, thereby maintaining user engagement and minimizing bounce rates.
- **Intuitive User Experience (UX):** The system aims to provide a clean, "zero-learning-curve" interface. Success in this area will be validated through usability testing with peer groups to ensure tasks like T-shirt customization can be completed without external assistance.

1.3.3. Success Metrics and Acceptance Criteria

The success of the Cre8tify prototype will be measured by the following acceptance criteria:

- **Complete Customer Journey Validation:** The system must facilitate a flawless transition from product browsing (e.g., the "Women's Collection") to design customization and, finally, to the generation of a valid "Order Confirmation."

- **Designer Portfolio Management Efficiency:** Creators must be able to upload artwork, set pricing, and view their designs in a "Pending" state within the dashboard. The system must also accurately reflect simulated sales and earnings data under a "My Sales" analytics module.

- **Administrative Quality Control:** The platform must empower administrators with absolute control over content visibility. This includes the ability to approve or reject designs based on quality and community guidelines, with status changes reflecting on the public store in real-time.

Chapter 02: System Analysis

2.1. Facts Gathering Techniques

To ensure the proposed system accurately addresses the needs of the Sri Lankan creative market, a multi-faceted fact-gathering approach was utilized. Since the project was inspired by personal technical barriers, the research focused on qualitative and secondary data collection:

- **Document Review & Secondary Research:** Extensive research was conducted on global E-commerce trends and the growth of the "Print-on-Demand" (POD) industry. Academic reports and market data from sources such as Technavio and ECDB were analyzed to establish the financial viability of the custom apparel market in Sri Lanka.

- **Competitive Benchmarking (System Audit):** A thorough "hands-on" analysis was performed on existing global platforms, including **Printify, Redbubble, and Teespring**. By interacting with these systems as a Sri Lankan user, it was possible to document specific friction points such as currency barriers, complex integration requirements (Etsy/Shopify), and the lack of localized payout methods.

- **Personal Observation & Experience:** As a digital designer, the researcher utilized an "Auto-ethnographic" approach, documenting personal challenges faced when attempting to commercialize designs from Sri Lanka. This provided "insider" data on the high cost of entry and the frustration of managing non-localized payment gateways.

- **Interface Heuristic Evaluation:** Existing local Sri Lankan apparel sites (e.g. Ozbek) were analyzed to evaluate their user interface and design-preview capabilities. This helped identify the "Visualization Gap" where current local systems fail to provide interactive design tools for buyers.

2.2. Existing System

The current landscape for custom apparel design is dominated by two types of systems:

1. Global Print-on-Demand Platforms (Printify, Redbubble, Teespring): These are highly sophisticated ecosystems that automate the design-to-print pipeline. They offer extensive catalogs and professional design tools. Users can upload artwork, set profit margins, and sync with global marketplaces. However, they operate on a "Global-First" model, which means their infrastructure is built for users with access to Stripe, PayPal, and international shipping logistics.

2. Localized Traditional E-commerce & Social Media Vendors (Ozbek, Instagram Shops): In Sri Lanka, the market is currently served by local brands like Ozbek or smaller independent vendors on Instagram and Facebook. While these provide local payment methods (Bank Transfers/Cash on Delivery), they lack automation. The "designing" happens via chat messages or email, and there is no real-time tool for the user to see a live preview of their custom text or graphics on the garment before the order is placed.

2.3. Drawbacks of the Existing System

The analysis of the systems mentioned above reveals several critical drawbacks for the target Sri Lankan audience:

- **Financial Exclusion:** The reliance on PayPal and credit-card-based payout systems effectively excludes Sri Lankan designers who do not have international merchant accounts.
- **Complexity and High Entry Barriers:** Platforms like Printify require users to manage a "middle-man" store (like Etsy), which involves listing fees and complex setup processes that discourage beginners.
- **Static Visualization:** Most local systems provide no interactive feedback. A buyer cannot see how a specific font or color looks on a T-shirt until after the physical product is delivered, leading to a "Buyer's Remorse" and high return rates.
- **Lack of Direct Designer Monetization:** There is no local hub where a designer can simply upload a design and get paid directly by a local buyer in LKR without dealing with foreign exchange complexities or high platform commissions.

2.4. User Persona Analysis

2.4.1. Persona 1: The Aspiring Designer (The Seller)

- **Name:** Kavindi, 21.
- **Background:** A Fine Arts student in Colombo who creates amazing digital illustrations but has no technical "web development" knowledge.
- **Goals:** Wants to see her art on high-quality T-shirts and earn a side income without paying monthly fees.

- **Frustrations:** Tried Printify but couldn't set up an Etsy store or get her money out via PayPal.
- **How Cre8tify helps:** Provides a free "Designer Dashboard" where she can upload her art, set a price, and get paid directly to her local bank account.

2.4.2. Persona 2: The Design Enthusiast (The Buyer)

- **Name:** Arshad, 24.
- **Background:** A young professional who hates wearing mass-produced clothes from big malls. He wants unique streetwear that matches his vibe.
- **Goals:** Wants to find unique designs and customize them (e.g. adding his name or changing the font color) to make them one-of-a-kind.
- **Frustrations:** Most local sites only show a flat image of the shirt, he’s afraid the design will look too small or in the wrong place when it arrives.
- **How Cre8tify helps:** The **Live Preview** tool allows him to see exactly how the design looks on a model, giving him the confidence to buy.

2.4.3. Persona 3: The System Administrator (Internal User)

- **Name:** Chamath, 30.
- **Background:** An e-commerce manager who needs to ensure the platform stays professional.
- **Goals:** To review dozens of daily submissions of designs in order to ensure that no low quality or inappropriate designs go live on the site.
- **Frustrations:** Manually reviewing every design via email or chat on a daily basis.
- **How Cre8tify helps:** The **Admin Panel** allows him to Approve or Reject designs instantly while keeping the store high-quality and safe.

Chapter 03: Requirement Specification

3.1. Functional Requirements

Functional requirements describe the specific services and behaviors the Cre8tify system must perform to satisfy the needs of its stakeholders. These are categorized by the three primary user roles defined in the system architecture.

3.1.1. Customer and User Interface Requirements

The primary goal for the general user is a seamless path from discovery to purchase. The system shall allow users to register and authenticate their identity securely. Once logged in, the application provides a robust browsing engine where users can filter through existing design collections. A critical functional requirement is the **Interactive Customization Engine**, which allows users to not only view but also personalize designs (adjusting text, colors, and positioning). This is supported by the **Live Preview and 360° Visualization Module**, which utilizes image processing to allow users to visualize the t-shirt on a dummy model or an uploaded front-facing photograph of themselves. Finally, the system must facilitate a complete checkout process, including cart management and purchase confirmation.

3.1.2. Designer Module Requirements

The platform serves as a creative workspace for local artists. The system shall provide a suite of free, browser-based design tools that enable designers to create artwork directly on the platform or upload pre-made digital assets. Designers must have the functional capability to edit their existing works, set competitive pricing in LKR, and manage their **"Digital Shop"** storefront. A dedicated designer dashboard is required to track the status of submissions and monitor sales analytics in real-time.

3.1.3. Administrative and Oversight Requirements

The Administrative role is essential for platform integrity. The system shall empower the Admin to manage and filter both designer and user accounts. A mandatory workflow is the **Design Moderation Queue**, where the Admin can approve or reject submitted designs based on quality and community standards. Furthermore, the system must allow the Admin to verify payment proofs (such as bank transfer slips) and monitor overall site activity to ensure a secure and civil marketplace.

3.2. Non-functional Requirements

Non-functional requirements describe how a system operates and provide criteria by which to judge these operations. They help ensure that the platform meets the needs of professional, reliable, and accessible operation.

- **Performance and Efficiency:** The system should load web pages and display intricate graphical designs in 5 seconds or less to encourage users to return.
- **Security and Data Integrity:** The system must implement industry-standard encryption for secure login and session management. Protecting the designers' creative works and buyers' private information is a top priority.
- **Scalability and Concurrency:** The backend architecture of the system must be optimized to handle up to 100 concurrent users without degradation in performance or system crashes, allowing for future growth in the Sri Lankan market.
- **Usability and Aesthetics:** The user interface (UI) of the application must be simple, user-friendly, and visually appealing. Following modern UI/UX principles, the system aims for a "zero-training" requirement, where even beginner designers can navigate the toolbox intuitively.
- **Portability and Responsiveness:** As a web-based platform, it must be fully responsive to mobile devices, maintaining a consistent and functional layout across desktop, laptop, tablet, and smartphone screens.

3.3. Hardware/ Software Requirements

The development and deployment of Cre8tify require a specific suite of hardware and software to ensure the MERN stack (MongoDB, Express, React, Node.js) and the image processing libraries function efficiently.

3.3.1. Development Environment

For the development phase, the hardware must be capable of running a local server, a database instance, and a modern IDE simultaneously. The minimum recommended specifications include:

- **Processor:** Intel Core i5 (10th Gen) or AMD Ryzen 5 equivalent.
- **Memory (RAM):** 8GB (16GB recommended for smoother rendering of canvas elements).
- **Storage:** 256GB SSD for rapid file indexing and build processes.

The software stack includes **Visual Studio Code** as the primary Integrated Development Environment (IDE), **Git** for version control, and **Postman** for API testing. The backend is powered by **Node.js**, while **MongoDB Atlas** is utilized for cloud-based data storage.

3.3.2. Client-Side (User) Requirements

On the consumer end, Cre8tify is designed to be accessible on any modern device.

- **Software:** Users need an evergreen web browser (Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge) and JavaScript and WebGL enabled for the 2D and 3D rendering engines.
- **Hardware:** If the user has purchased a smartphone or PC in the past five years and has 2 GB of RAM available to run the application, they should be able to use it without any issue.

3.4. Technical Constraints and Image Specifications

Unlike a standard e-commerce site, Cre8tify faces unique technical constraints due to its heavy reliance on client-side image manipulation.

- **Image Fidelity and Resolution:** To ensure that designs created on the platform are suitable for physical printing, the system enforces a constraint on image resolution. Uploads must meet a minimum DPI (Dots Per Inch) threshold to prevent pixelation during the "Live Preview" and final production stages.

- **CORS and Canvas Security:** A major technical constraint of Cre8tify application is **Cross-Origin Resource Sharing (CORS)**. When the design tool fetches images from the cloud (e.g. Cloudinary or S3), the browser's "Canvas" element can become tainted for security reasons. The system must implement specific CORS headers and crossOrigin = "anonymous" attributes to allow users to download their final designs or generate mockups.

- **Browser Memory Limits:** Handling multiple high-resolution layers (images, text, and vectors) simultaneously on the React state can lead to browser tab crashes on low-end mobile devices. The system is constrained to optimize memory by flattening layers during the preview process.

Chapter 04: Feasibility Study

4.1. Operational Feasibility

Operational feasibility evaluates how well the proposed system solves the identified problems and whether it will be embraced by its target users. For Cre8tify, this is high due to the localized nature of the solution.

The system is designed with a "zero-learning-curve" philosophy. For **Designers**, the operational flow mimics familiar social media upload patterns, making it accessible to creative individuals who may lack deep technical expertise. By providing an integrated toolbox, the platform eliminates the need for expensive external software, which is a major operational barrier for local students.

For **Buyers**, the feasibility is rooted in "trust-building." In the current Sri Lankan market, online shopping anxiety is high due to the "Expectation vs. Reality" gap. The implementation of the **Live Preview** tool directly addresses this by allowing users to visualize products before purchase. Furthermore, by using local terminology and simplified navigation, the platform ensures that users of varying tech-literacy levels in Sri Lanka can successfully complete a transaction.

4.2. Economical Feasibility

This section analyzes the cost-effectiveness of the project. The fact that the Cre8tify platform is both economically and technologically feasible is largely due to the fact that it is built on an **Open-Source** technology stack **(MERN),** eliminating the high license fees associated with proprietary software.

- **Development Costs:** By using JavaScript across the entire stack (React and Node.js), development time is reduced, and the need for a large team with multiple language skills is eliminated.
- **Operational Costs:** When local businesses require manual order processing and there are mistakes made, they lose out on a large amount of revenue. The Cre8tify platform will automate the process from design to order and will reduce both the possibility of human error as well as the amount of administrative work required of individuals.
- **Market Opportunity:** As identified in Chapter 01, the custom apparel market in Sri Lanka is growing by 15-20% annually. Unlike global platforms that take high commissions and require USD subscriptions, Cre8tify offers a low-barrier entry for local creators, making it a sustainable "local-first" business model. The transition to local bank transfers also removes the hidden currency conversion fees often lost during international transactions.

4.3. Technical Feasibility

Technical feasibility assesses whether the current technology is capable of supporting the project's complex requirements, particularly the image-heavy design tool.

The choice of the **MERN Stack** (MongoDB, Express, React, Node.js) provides a robust foundation for this project.

- **React.js (Frontend):** Is technically ideal for building the Design Toolbox. Its component-based architecture and Virtual DOM allow for high-performance updates when a user moves, rotates, or scales an image layer on the T-shirt.

- **Node.js & Express (Backend):** These provide a non-blocking, event-driven environment that is highly efficient for handling multiple concurrent requests, such as when many users are uploading high-resolution designs simultaneously.

- **MongoDB (Database):** Unlike traditional SQL databases, MongoDB's document-based structure is perfectly fit for storing "T-shirt Designs." Each design can be stored as a JSON-like object containing dynamic arrays of layers, coordinates, and font styles, making the data retrieval process extremely fast.

- **Third-Party Libraries:** The technical success is further ensured by the availability of mature libraries like html2canvas for generating previews and ReactCrop for image manipulation.

In conclusion, the existing hardware and software infrastructure are more than sufficient to support the development and scaling of the Cre8tify platform.

Chapter 05: System Architecture

5.1. Use Case Diagram

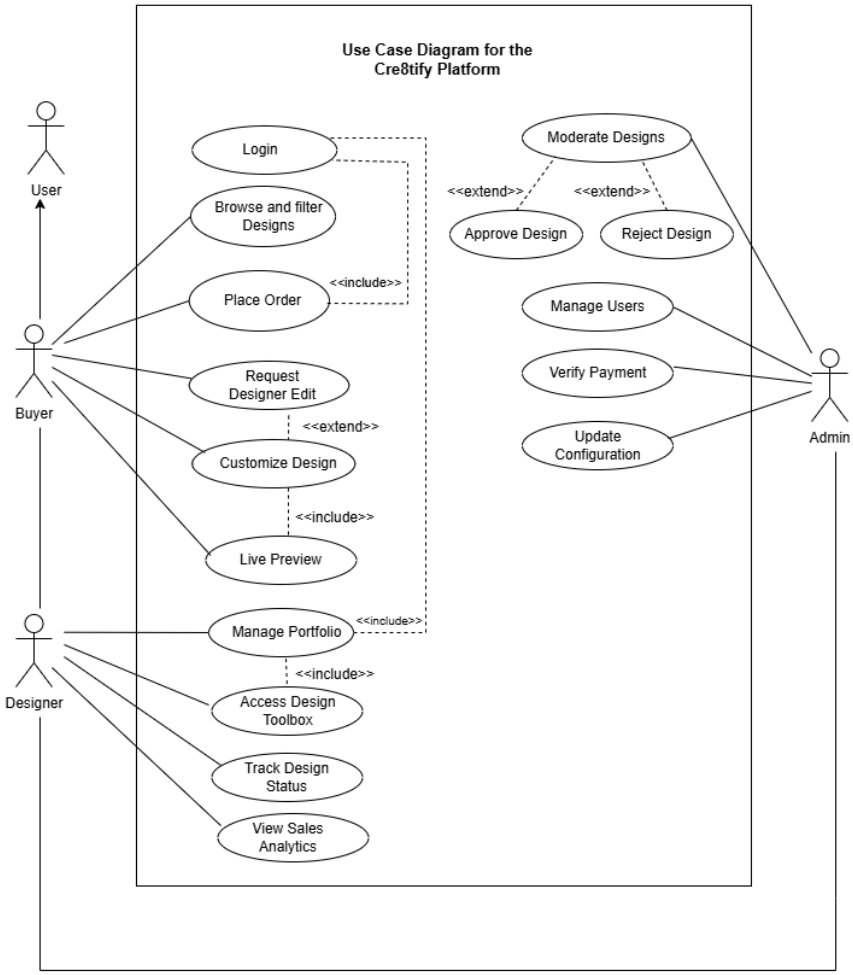

Figure 1: Use Case Diagram

The Use Case Diagram illustrates the functional interactions between users and the Cre8ify platform. The system consists of three primary actors: **Buyer (Customer), Designer, and Admin**. Each actor interacts with the system based on role-specific responsibilities.

The Buyer can browse and filter designs, customize selected designs, preview modifications, request designer edits, and place orders. The “Place Order” use case includes “Login,” as authentication is required before completing a purchase. The “Request Designer Edit” use case extends “Customize Design,” indicating optional behavior.

The Designer manages their portfolio, accesses the design toolbox, tracks design status, and views sales analytics. The Admin oversees system operations by moderating designs (approve or reject), managing users, verifying payments, and updating platform configurations.

The diagram distinguishes between mandatory and optional behaviors using <<include>> and <<extend>> relationships, providing a clear representation of system functionality and user responsibilities.

5.2. Class Diagram

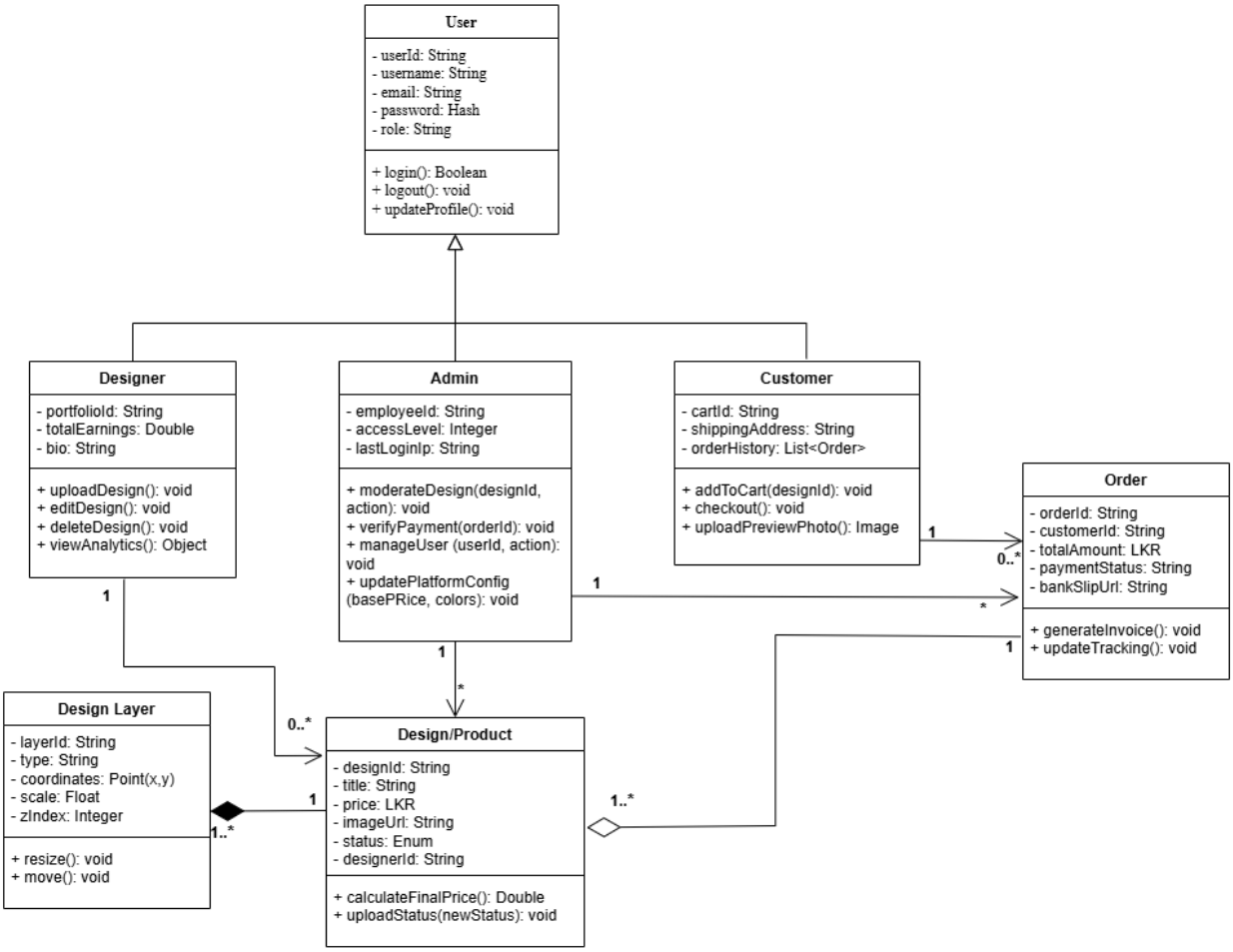

Figure 2: Class Diagram

The Class Diagram represents the structural design of the Cre8ify platform using object-oriented principles. The core class is User, which serves as a parent class for Designer, Customer, and Admin through inheritance. This approach promotes reusability and centralized authentication.

The User class includes attributes such as userId, username, email, password, and role. The Designer class extends User and manages design-related operations. The Customer class enables order placement and checkout functionality. The Admin class includes additional

capabilities for moderating designs, verifying payments, managing users, and updating system configurations.

The Design class represents products uploaded by designers, while DesignLayer represents individual editable components of a design. The Order class stores transaction details, linking customers to purchased designs.

The class diagram defines relationships between users, designs, and orders, providing a clear blueprint for system implementation and object interaction.

5.3. ER Diagram

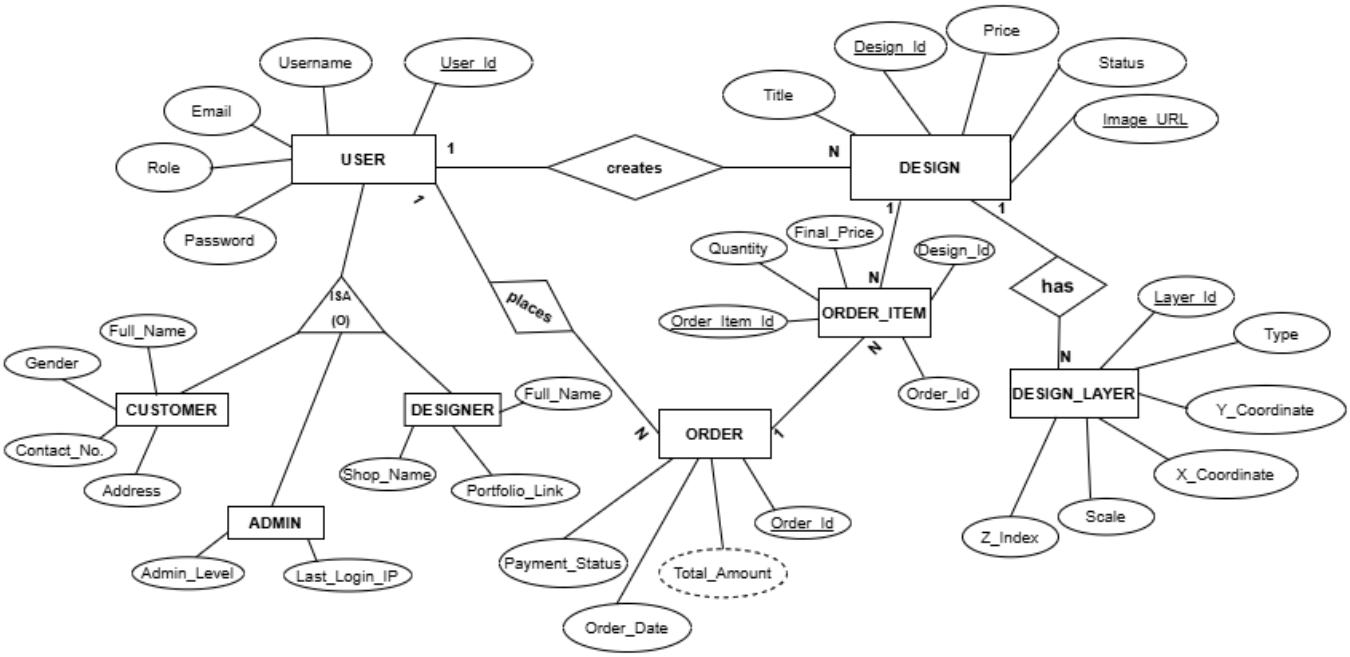

Figure 3: ER Diagram

The ER Diagram illustrates the logical data structure of the Cre8tify system using entities, attributes, and relationships. The primary entity is **USER**, identified by User_Id. The system uses overlapping specialization **(ISA),** allowing a user to function as a CUSTOMER and/or DESIGNER, while ADMIN is a restricted role. Each subtype contains role-specific attributes such as Contact_No and Address for CUSTOMER, Shop_Name and Portfolio_Link for DESIGNER, and Admin_Level and Last_Login_IP for ADMIN.

A USER can create multiple DESIGN entities (1:N relationship). Each DESIGN is identified by Design_Id and includes attributes such as Title, Price, Status, and Image_URL.

A DESIGN can contain multiple DESIGN_LAYER entities (1:N), where each layer stores positional and scaling details such as X_Coordinate, Y_Coordinate, Scale, and Z_Index.

A USER can place multiple ORDER entities (1:N relationship). Each ORDER, identified by Order_Id, contains attributes including Payment_Status, Order_Date, and a derived Total_Amount.

The many-to-many relationship between ORDER and DESIGN is resolved using the associative entity ORDER_ITEM. ORDER_ITEM includes Order_Item_Id, Order_Id (FK), Design_Id (FK), Quantity, and Final_Price. This structure ensures data integrity and accurately represents transactional relationships within the system.

5.4. High-Level Architectural Diagram

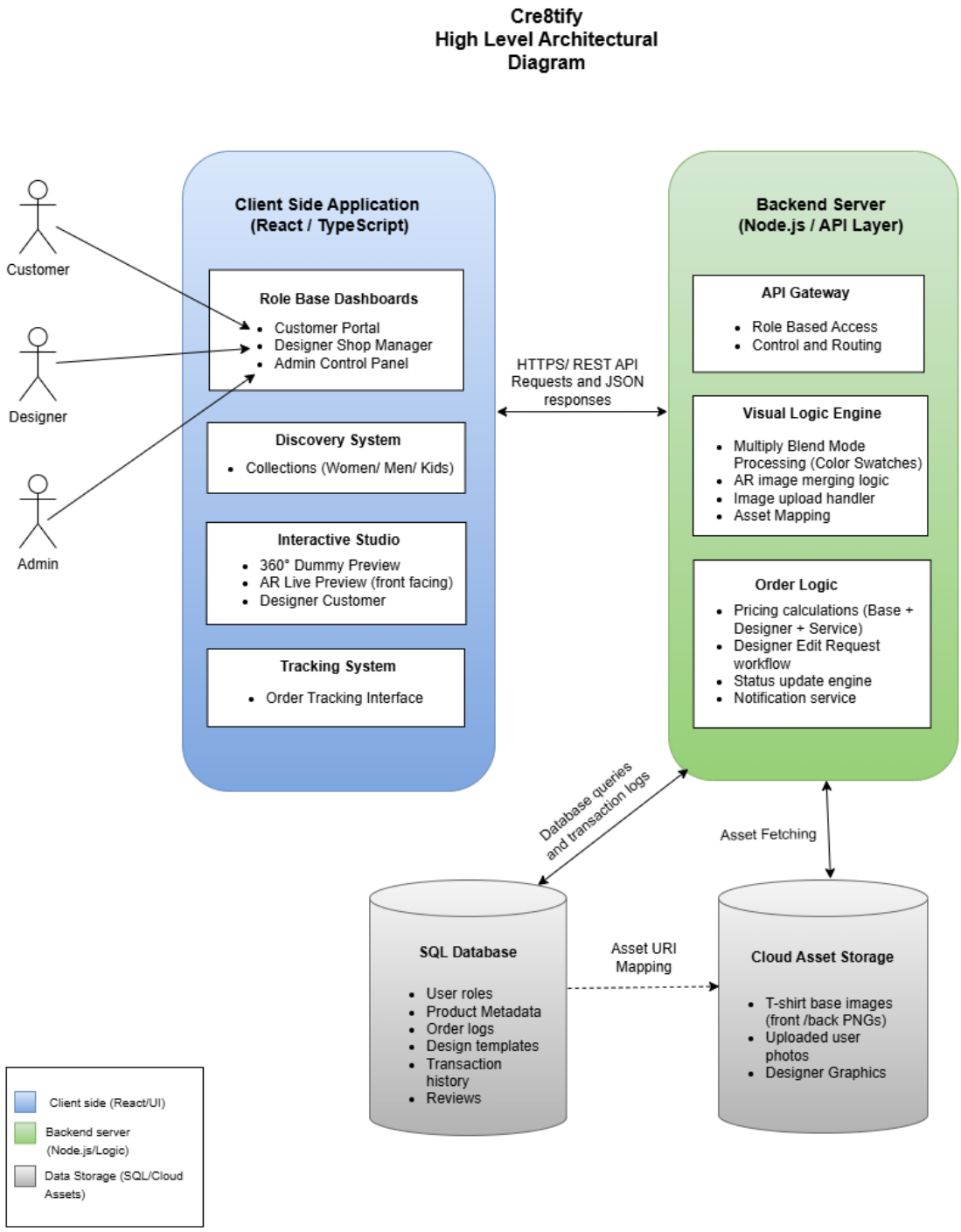

Figure 4: High Level Architectural Diagram

The High-Level Architectural Diagram illustrates the overall system structure and interaction between major components of Cre8tify. The system follows a layered web architecture which contains Client Layer, Backend Layer, Database Layer, and Cloud Asset Storage.

- **Client-Side Application:** developed using React and TypeScript. It provides role-based dashboards for Customers, Designers, and Admins. The client includes modules such as the Discovery System, Interactive Studio, and Order Tracking System. Users interact with the system through HTTPS REST API requests and receive JSON responses from the backend.

- **Backend Server:** developed using Node.js, contains the API Gateway, Visual Logic Engine, and Order Logic module.

- **API Gateway:** handles routing and role-based access control.

- **Visual Logic Engine:** manages image processing, layer merging, asset mapping, and upload handling.

- **Order Logic module:** processes pricing calculations, order workflows, status updates, and notifications.

The backend communicates with the **SQL Database**, which stores structured data such as user roles, product metadata, design templates, orders, transactions, and reviews. While database queries and transaction logs are handled securely through the server.

Additionally, the system integrates **Cloud Asset Storage** for storing T-shirt base images, uploaded user images, and designer graphics. The backend fetches assets when required and maps asset URIs for rendering in the interactive studio.

This layered architecture ensures scalability, maintainability, and separation of concerns. The client handles presentation logic, the server manages business logic, and the database and cloud storage manage persistent data and assets efficiently.

Chapter 06: Development Tools and Technologies

6.1. Development Methodology

The Agile Kanban approach has been chosen for Cre8tify's development. Unlike with a more traditional Waterfall Method, the flexibility of Agile allows iterative development. This is very important for a complex system using many refinements. The project will break down into 2-week “sprints” and will use a Kanban board to track tasks through 4 stages – **Backlog, In Progress, Testing and Completed.** This will provide for continued integration into the project and allow for rapid-return to a task if one particular library (like the canvas engine) has to change. This approach is well suited for a MERN stack application because it allows for concurrent development of the front-end and back-end.

6.2. Programming Languages and Tools

The core of Cre8tify is built on the **MERN Stack**, selected for its high performance and use of a single language (JavaScript/TypeScript) across the entire stack.

- **TypeScript:** Used as the primary language over vanilla JavaScript to provide static typing, which reduces runtime errors in the complex customization logic.
- **React.js (Frontend):** This was selected due to its component-based architecture and virtual DOM. The Virtual DOM is necessary for the ‘Live Preview’ functionality, where some of the components must re-render almost instantly as they change without refreshing the entire page.
- **Node.js & Express (Backend):** Node and Express are used to create a RESTful API. Node’s non-blocking I/O model fits perfectly into handling multiple concurrent users during busy shopping hours.
- **MongoDB (Database):** A NoSQL database was selected for storage because the different t-shirt designs have several different properties (layers, colors, metadata), which will not fit nicely into the rigid nature of SQL database rows.
- **Visual Studio Code:** The primary Integrated Development Environment (IDE), utilizing extensions like ESLint and Prettier to maintain code quality.

6.3. Third-Party Components and Libraries

To avoid "reinventing the wheel" and to ensure professional-grade functionality, several specialized libraries are integrated:

- **Fabric.js / HTML5 Canvas:** This is the most critical library. It provides the interactive canvas for the Design Toolbox, allowing users to drag, scale, and rotate text and images.
- **Redux Toolkit:** Used for state management. It tracks the user’s "Cart" and "Design Progress" globally across the application.

- **Cloudinary API:** Used for image hosting. Instead of storing heavy design files on the local server, Cloudinary provides cloud-based storage and automatic image optimization.
- **Axios:** For making asynchronous HTTP requests from the React frontend to the Node.js backend.
- **Multer:** A middleware for handling multipart/form-data, used specifically for uploading t-shirt design files and bank payment slips.

6.4. Algorithms

The system utilizes two primary algorithmic logic patterns to ensure efficiency:

1. **Price Calculation Algorithm:** A custom function that calculates the final product price based on the base t-shirt cost + designer commission + service charge + tax.

   $$Total = (B + C) * (1 + S)$$

   Where B is the base price, C is the designer's fee, and S is the service charge percentage.

2. **Layer Rendering Algorithm:** When a user saves a design, the system converts the JSON state of the Fabric.js canvas into a high-resolution PNG. This involves a **Z-Index Sorting Algorithm** to ensure that text and images are layered correctly (bottom to top) before the final merge.

Chapter 07: Implementation Progress

7.1. Development Environment Setup

The development environment for Cre8tify was established to ensure a seamless transition between frontend and backend workflows. The following configurations were implemented:

- **Node.js Runtime:** Version 18.x LTS was installed to ensure compatibility with the latest React and Express features.
- **Version Control:** To enable a systematic workflow for managing code change documents, we have made a private repository with **GitHub** using a branch structure (Main Branch, Dev Branch and Dev Feature Branches) that can be used to develop code changes.
- **Database Configuration:** We have created a MongoDB Atlas cluster using cloud resources to provide a database for storing app data so that we can develop the application through the cloud and access the database from our computers and through the application development stage.
- **API Testing Tools: Postman** was configured to test RESTful endpoints before integrating them with the React frontend.
- **Environment Variables:** A **.env** file structure was established to protect sensitive data such as the MongoDB URI and JWT Secret Keys, following industry security standards.

7.2. Implemented Features

As of this interim stage, the core "skeleton" of the Cre8tify platform has been successfully developed. The following features are currently functional:

1. **Authentication System:** A robust User Registration and Login system using **JWT (JSON Web Tokens)** for secure session management.
2. **Product Marketplace:** A dynamic grid that fetches and displays approved designs from the MongoDB database.
3. **Basic Design Canvas:** A prototype of the **Fabric.js** toolbox allowing for basic text manipulation and color changes on a t-shirt template.
4. **Admin Dashboard (Partial):** A secure interface where an admin can view a list of pending design submissions.
5. **User Profile Management:** The ability for users to update their bio and view their previous activity.

7.3. Screenshots / Code Snippets

7.3.1. Database Schema and Persistence (MongoDB Compass)

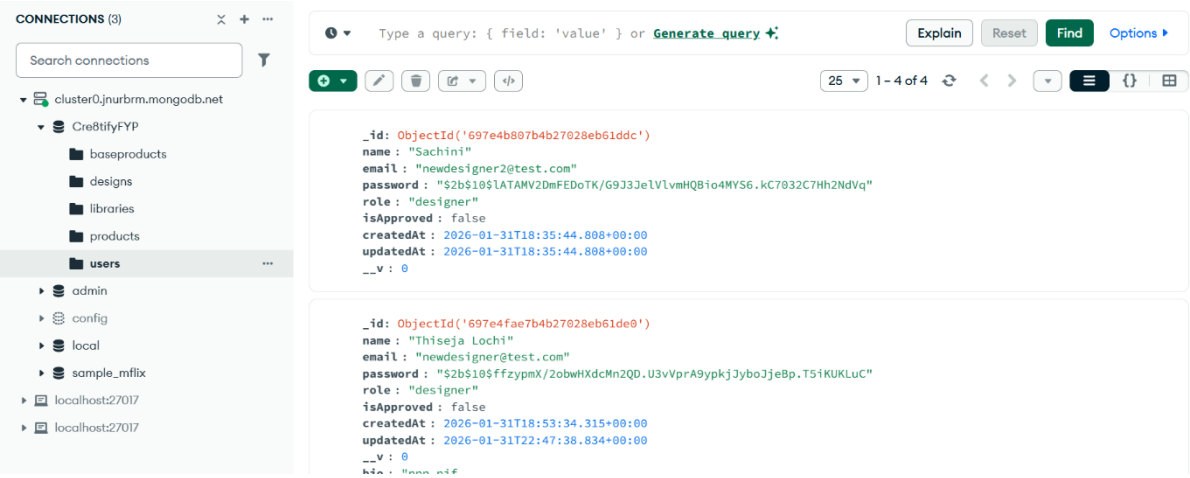

Figure 5: MongoDB Compass

This illustrates the live state of the **Cre8tify** database using MongoDB Compass. This screenshot provides empirical evidence of successful data persistence and schema validation. As observed, the 'users' collection correctly implements the model defined in Chapter 05, featuring unique ObjectIDs, hashed passwords for security, and role-based categorization (e.g., 'designer'). The inclusion of automated timestamps (*createdAt* and *updatedAt*) further confirms the successful integration of Mongoose middleware, ensuring a reliable audit trail for all user-generated data.

7.3.2. Security Layer: Authentication Middleware (authMiddleware.js)

```
cre8tify > backend > middleware > JS authMiddleware.js > ...
const jwt = require('jsonwebtoken');
const asyncHandler = require('express-async-handler');
const User = require('../models/User');

// 1. Protect Middleware: Checks for a valid JWT token
const protect = asyncHandler(async (req, res, next) => {
    let token;
    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        try {
            token = req.headers.authorization.split(' ')[1];
            const decoded = jwt.verify(token, process.env.JWT_SECRET);
            // User info (excluding password) is now in req.user
            req.user = await User.findById(decoded.id).select('-password');
            next();
        } catch (error) {
            console.error(error);
            res.status(401);
            throw new Error('Not authorized, token failed');
        }
    }
    if (!token) {
        res.status(401);
        throw new Error('Not authorized, no token');
    }
});
```

Figure 6: Security Layer

The implementation of the protect middleware, as shown in the snippet above, serves as the primary security gateway for the Cre8tify API. By intercepting incoming requests and verifying the **JSON Web Token (JWT)** within the Authorization header, the system ensures that sensitive endpoints remain inaccessible to unauthorized users. This logic not only decodes the token to identify the requesting user but also attaches the user's profile to the request object (req.user), allowing for seamless permission checks in subsequent controller functions.

7.3.3. Data Modeling: User Entity (User.js)

```
cre8tify > backend > models > JS User.js > ...
1   const mongoose = require('mongoose');
2
3   const userSchema = mongoose.Schema(
4     {
5       name: { type: String, required: [true, 'Please add a name'], },
6       email: { type: String, required: [true, 'Please add an email'], unique: true,},
7       password: { type: String, required: [true, 'Please add a password'],},
8       role: { type: String,required: true,enum: ['buyer', 'designer', 'admin'], default: 'buyer',},
9
10      // --- FIELDS FOR CUSTOMERS ---
11      phone: { type: String,},
12      address: { type: String,},
13      gender: {type: String,},
14      interest: {type: String,},
15
16      // --- ● NEW FIELDS FOR DESIGNERS ---
17      brandName: { type: String, },
18      bio: { type: String, },
19
20      isApproved: {
21        type: Boolean,
22        default: false,
23      }
24    },
25    {
26      timestamps: true,
27    }
28  );
29
30  module.exports = mongoose.model('User', userSchema);
```

Figure 7: Data Modelling

The User Model defines the blueprint for identity management within the system. By utilizing an **Enum** for user roles (Buyer, Designer, Admin), the architecture enforces strict access control from the database level upward. This snippet also highlights the use of the unique constraint on email addresses, which prevents account duplication and ensures data integrity. This model serves as the foundation for the system's multi-tenant environment, allowing different user types to access specialized dashboards.

7.3.4. RESTful Routing Architecture (designRoutes.js)

```
cre8tify > backend > routes > JS designRoutes.js > ...
const express = require('express');
const router = express.Router();
const { protect, designer } = require('../middleware/authMiddleware');
const designCtrl = require('../controllers/designController');

// Import Multer upload middleware
const upload = require('../middleware/uploadMiddleware');

// Public Routes: Accessible by all users
router.route('/').get(designCtrl.getAllApprovedDesigns);
router.route('/:id').get(designCtrl.getDesignById);

// Private Designer Routes
router.route('/')
    .post(protect, designer, upload.single('image'), designCtrl.createDesign);

// Designer Management: Fetching and deleting specific user designs
router.get('/my-designs', protect, designer, designCtrl.getDesignsByDesigner);
router.delete('/:id', protect, designer, designCtrl.deleteDesign);

module.exports = router;
```

Figure 8: RESTful Routing Architecture

showcases the centralized routing logic for the Design module. This code demonstrates a modular approach to API development, where public routes (fetching designs) are separated from private CRUD operations. A critical technical detail is the **middleware pipeline** applied to the POST and DELETE routes. By chaining authentication, role-check, and Multer file-upload middleware, the system ensures that design assets are processed securely and only by authorized creators, maintaining a high standard of platform security.

7.3.5. Business Logic: Design Controller (designController.js)

```
cre8tify > backend > controllers > JS designController.js > createDesign > asyncHandler() callback
14  const createDesign = asyncHandler(async (req, res) => {
15    const { title, description, price } = req.body;
16
17    // The image path is created by Multer and stored in req.file.path
18    const imageUrl = req.file ? req.file.path : null;
19
20   // 1. Validation for form fields AND the file upload
21   if (!title || !description || !price || !imageUrl) {
22
23       // If validation fails but a file was uploaded, clean up the file
24        if (imageUrl) {
25          try {
26              // FIX: Use synchronous unlink to simplify and ensure immediate cleanup
27          fs.unlinkSync(imageUrl);
28          } catch (error) {
29          console.error(`Failed to delete failed upload file: ${imageUrl}`, error);
30        }
31        }
32
33      res.status(400);
34      throw new Error('Please include title, description, price, and upload a design image.');
35   }
```

Figure 9: Business Logic

The **createDesign** controller represents the core functional logic of the design marketplace. Beyond simple data entry, this snippet illustrates advanced error-handling strategies, specifically the use of **fs.unlinkSync** to manage server storage. If a design submission fails validation, the system automatically deletes the 'orphaned' image file from the local directory. This ensures that the server remains optimized and free of redundant data, reflecting a professional approach to backend resource management.

7.3.6. Frontend Integration: User Login and State (Login.tsx)

```
    try {
        // 1. The Handshake: Talk to the Backend
        const response = await fetch('http://localhost:5000/api/users/login', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ email, password }), // Sending your typing to the server
        });

        const data = await response.json();

        if (response.ok) {
            // 2. The Memory: Save the user info and Token in the browser
            // This is how the Dashboard knows you are "Dulni"
            localStorage.setItem('userInfo', JSON.stringify(data));

            // 3. The Entry: Go to the correct dashboard based on the role from the Databas
            if (data.role === 'designer') {
                navigate('/designer-dashboard');
            } else {
                navigate('/buyer-dashboard');
            }
        } else {
            // If password is wrong or user doesn't exist
            alert(data.message || "Invalid Email or Password");
        }
    } catch (error) {
        console.error("Login error:", error);
        alert("Server error. Is your backend running?");
    }
};
```

Figure 10: Frontend Integration

The frontend login implementation facilitates the essential 'handshake' between the React client and the Express backend. The logic utilizes the **Fetch API** to transmit credentials and receives a JWT in return, which is then persisted in the browser's localStorage. Notably, this code handles dynamic navigation; by analyzing the user's role returned from the database, the system automatically redirects the individual to their respective Designer or Buyer dashboard. This ensures a personalized and secure user experience immediately upon authentication.

7.3.7. User Interface (UI) and Frontend Presentation

7.3.7.1. Landing Page

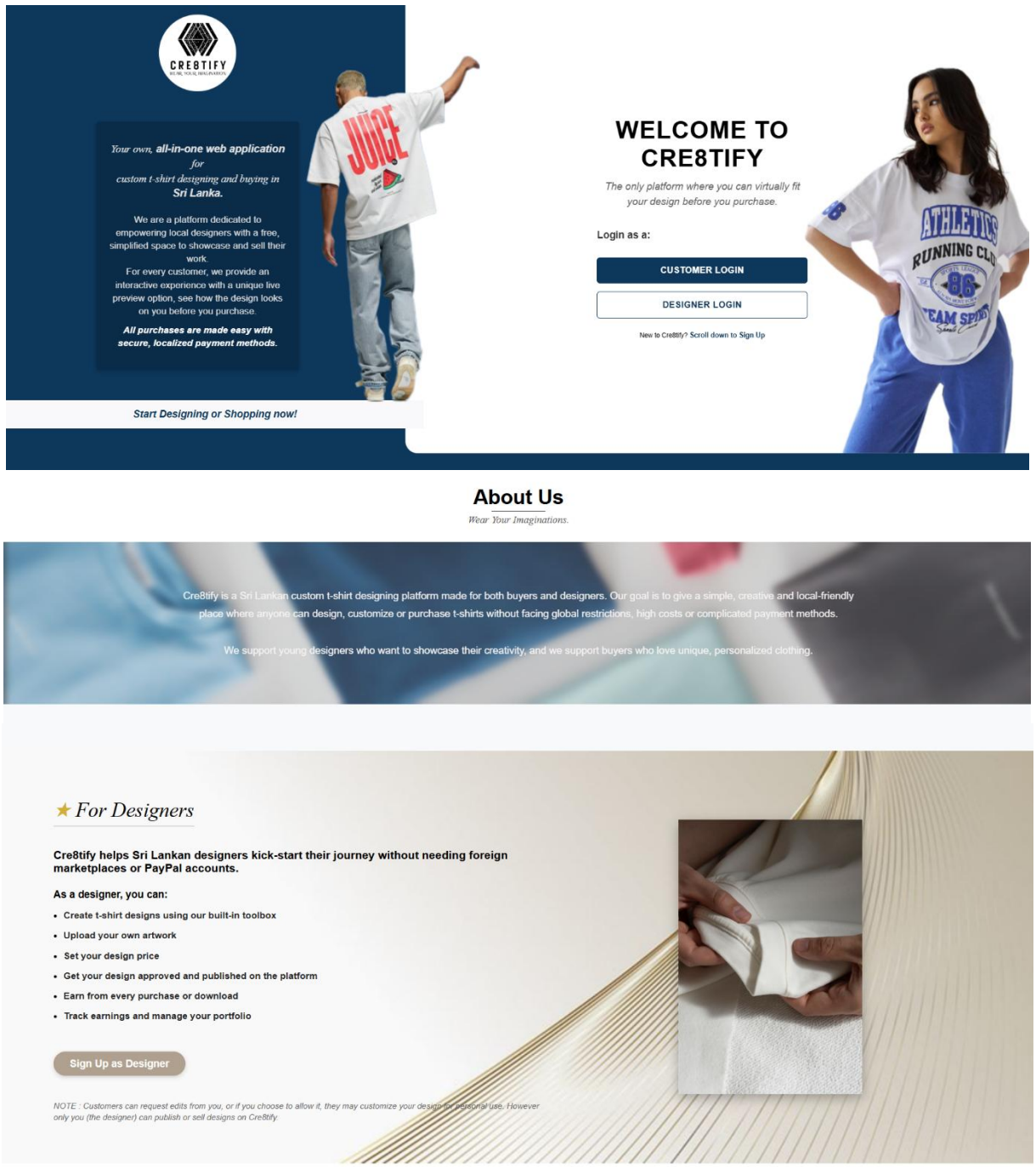

Figure 11: Landing Page

7.3.7.2. Designer Signup

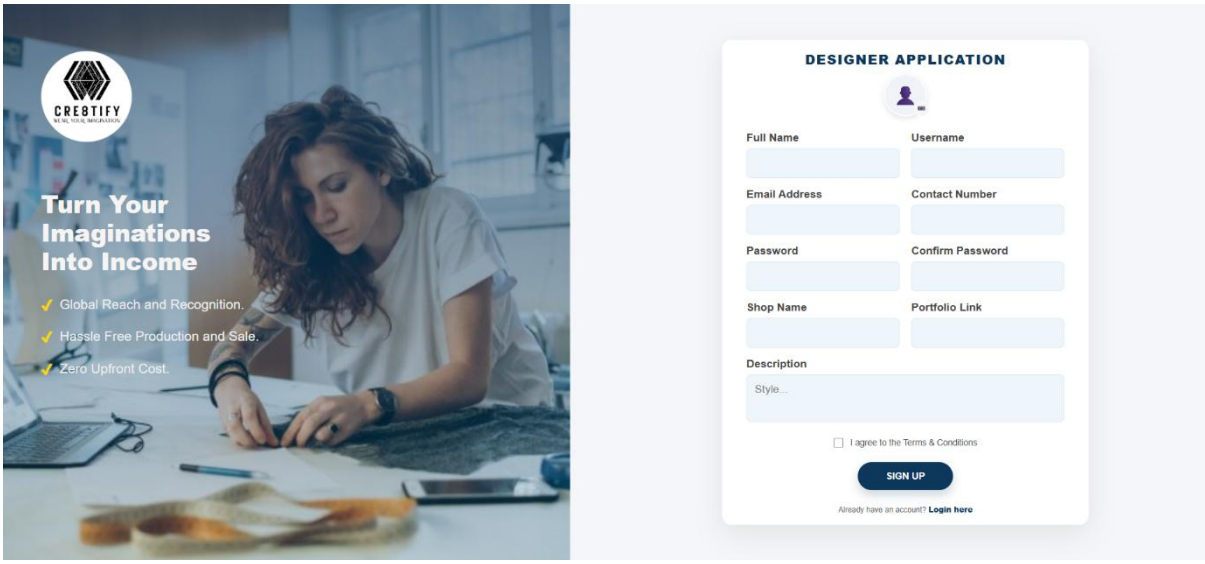

Figure 12: Designer SignUp

7.3.7.3. Designer Dashboard

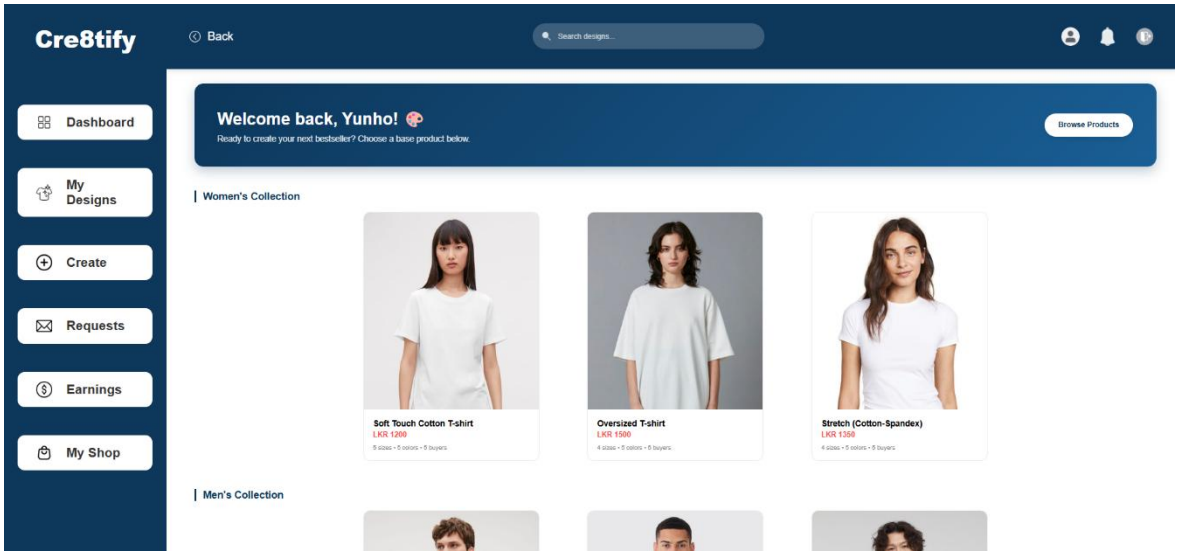

Figure 13: Designer Dashboard

7.3.7.4. Designing Page

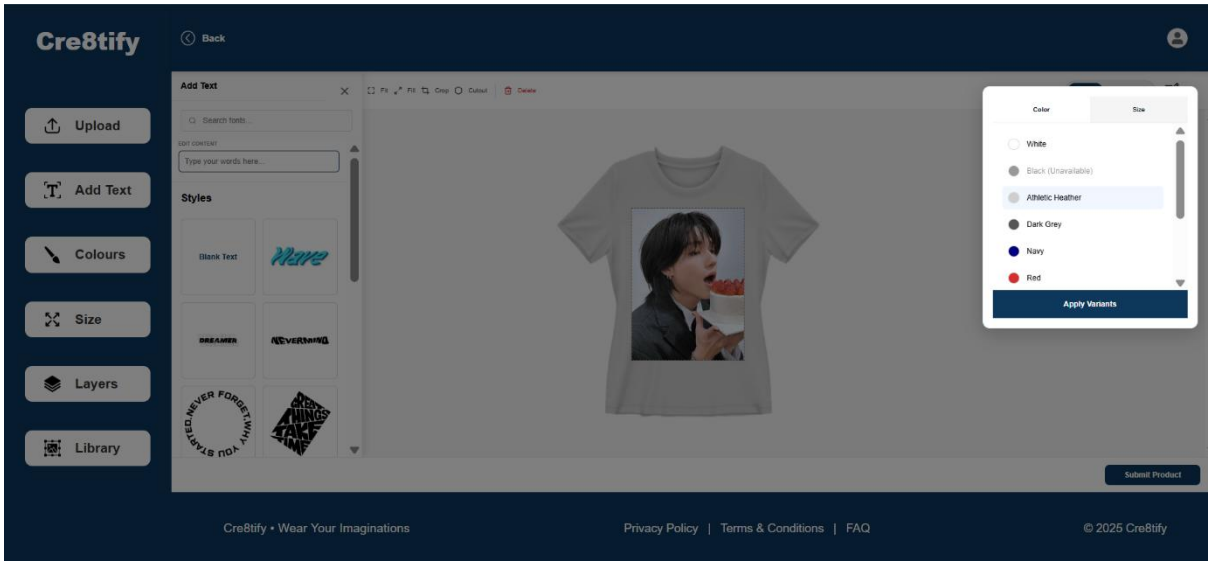

Figure 14: Designing Page

7.3.7.5. My Shop

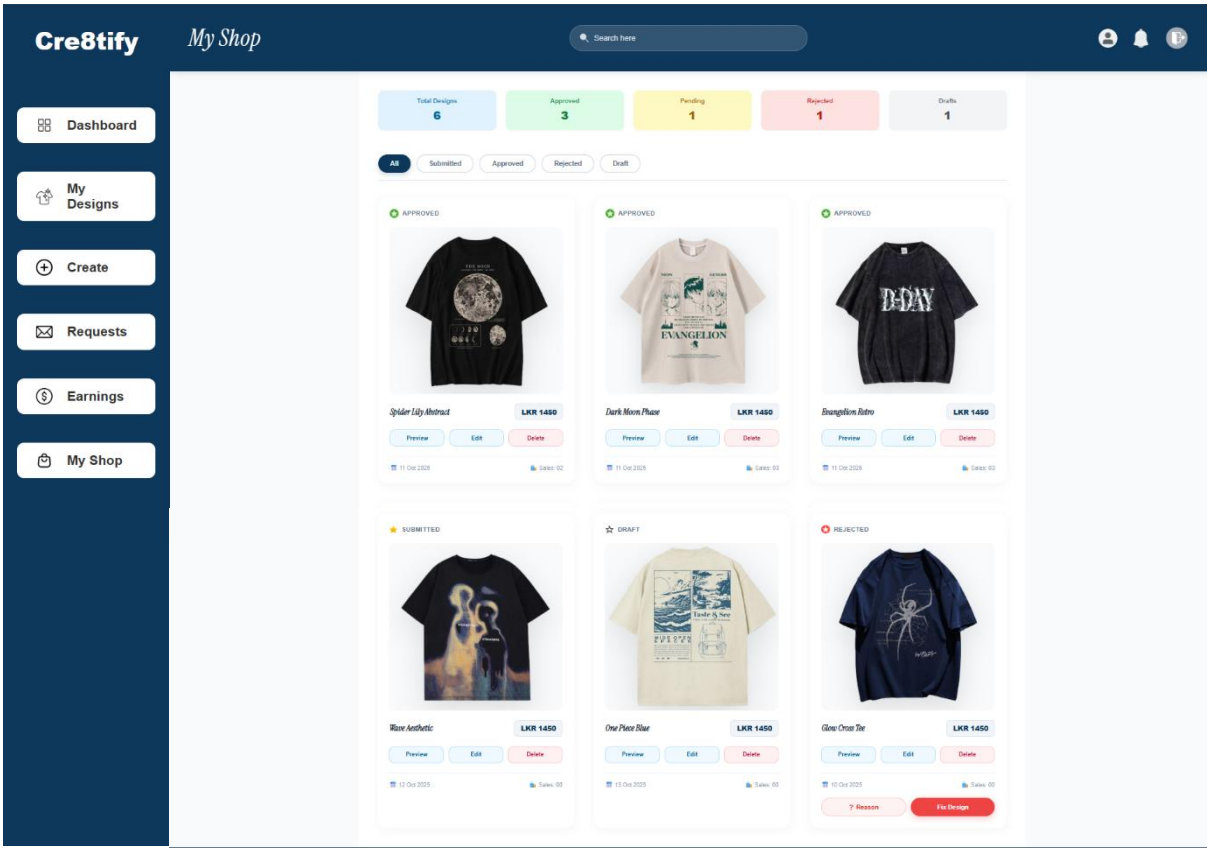

Figure 15: My Shop

7.3.7.6. Customer Dashboard

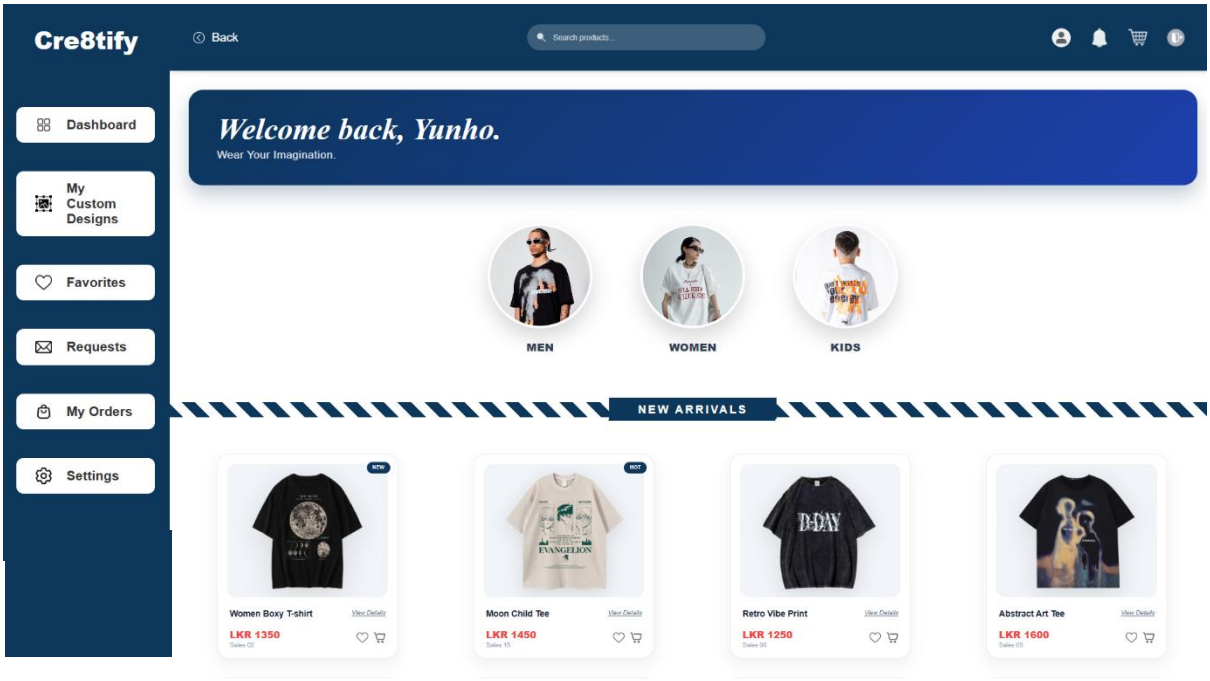

Figure 16: Customer Dashboard

7.3.7.7. Profile (Account Settings)

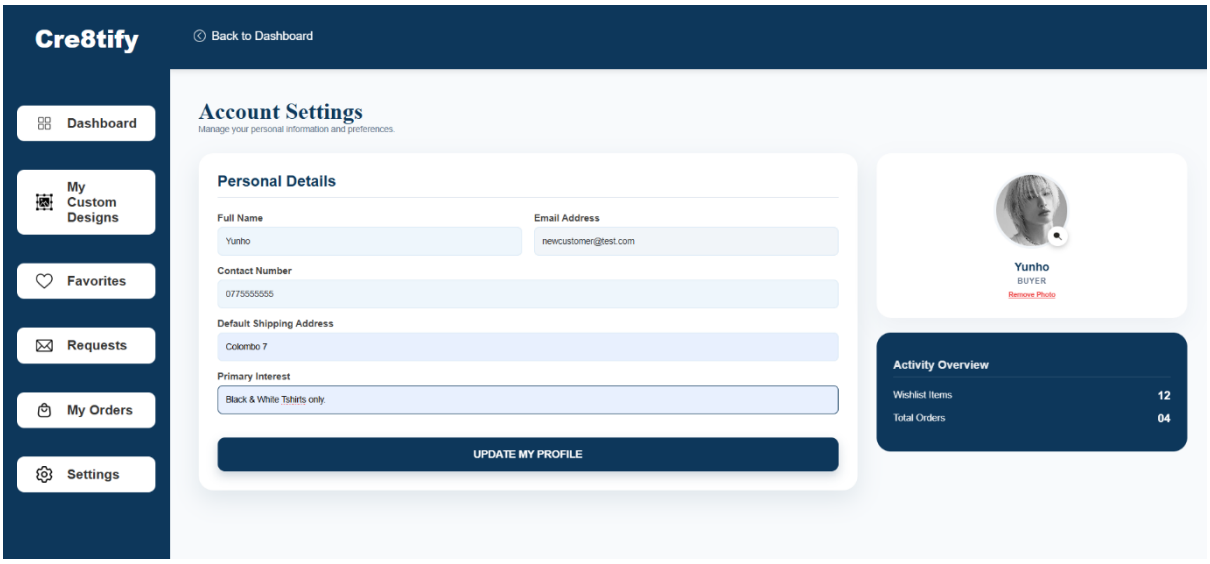

Figure 17: Profile

7.3.7.8. Women Collection

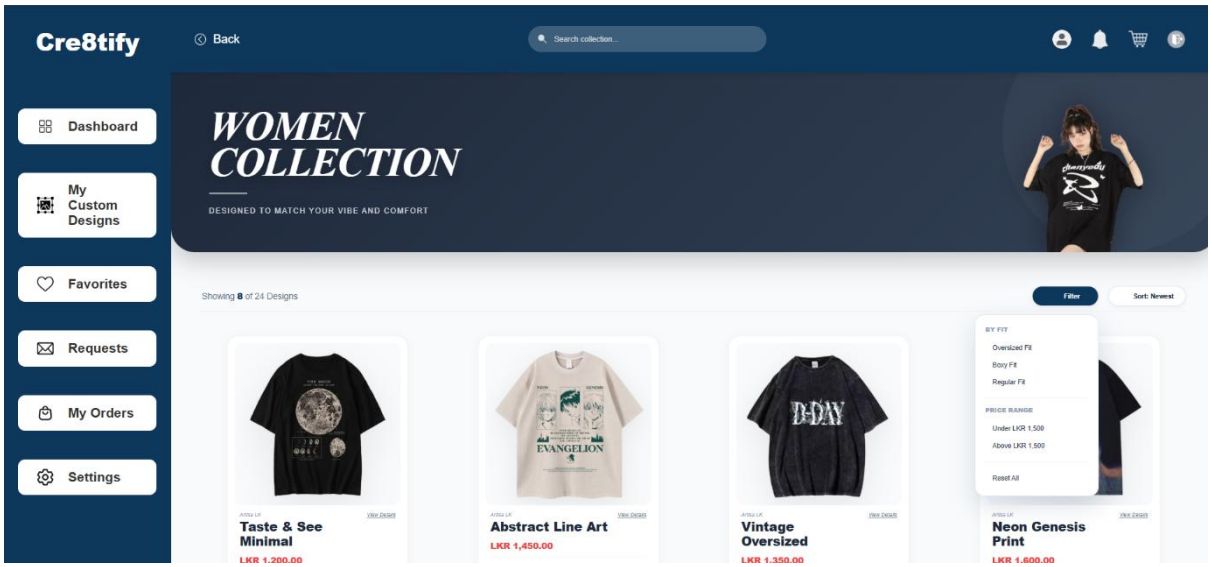

Figure 18: Women Collection

7.3.7.9. Live Preview

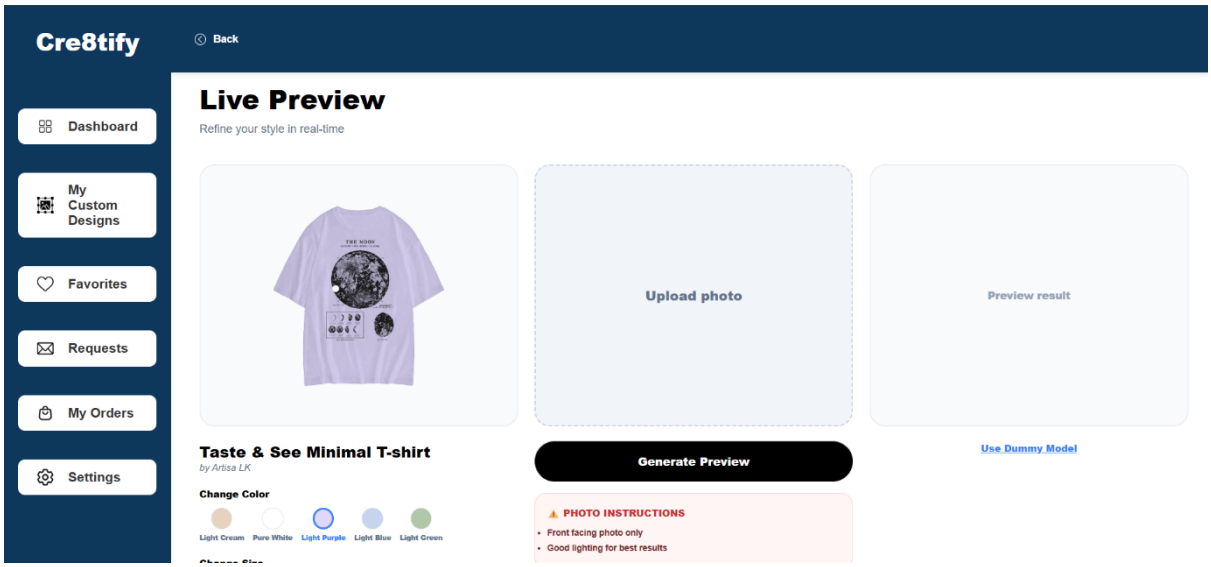

Figure 19: Live Preview

7.4. Challenges Encountered and Solutions

The implementation phase of the Cre8tify platform encountered a number of barriers in terms of technology and architecture; the table below lists the main issues and how they were resolved:

Challenge	Impact on Development	Technical Solution
Cross-Origin Resource Sharing (CORS)	The React frontend (Port 3000) was unable to communicate with the Express backend (Port 5000) due to browser security policies.	Configured the *cors* middleware in the Express entry file to whitelist the frontend origin, allowing secure header and token exchange.
State Persistence on Refresh	User authentication data was being lost whenever the browser page was refreshed, forcing redundant logins.	Implemented **LocalStorage** persistence combined with a *useEffect* hook in the root React component to validate tokens on mount.
Storage Bloat from Failed Uploads	Image files were being saved to the server even if the subsequent database entry failed due to validation errors.	Integrated *fs.unlinkSync* within the controller's catch block to delete orphaned files immediately upon a validation failure.
Complex Multi-Part Forms	Standard JSON requests could not handle binary image data alongside design metadata (title, price).	Transitioned to the **FormData API** on the client side to package textual and binary data for processing by the **Multer** engine.

Table 1

7.5. Current System Limitations

While the core authentication and design management modules are functional, the current interim version of Cre8tify has the following limitations:

- **Hybrid Payment Implementation:** While the system successfully integrates a **Sandbox** environment for simulated gateway transactions and supports manual **Bank Transfer** verification, currently it is limited to manual modes and testings. The transition to a production-ready, live API integration (handling real-currency transactions) is pending final security audits and business registration.

- **Static Design Previews:** Cre8tify currently uses a **2D overlay engine** with CSS effects like shadows to simulate a 3D aesthetic. With that, a research is ongoing to transition toward a true **3D mesh-mapping engine** (using libraries like Three.js) to allow designs to wrap dynamically around garment contours in the final version.

- **Real-time Notifications:** currently the UI includes a notification bell to alert designers regarding their design approvals, but these updates are not "real-time." The system currently requires a manual page refresh or a new navigation event to trigger a database query and update the notification status.

- **Manual Content Moderation:** currently the platform depends on human administrators for design approval, creating an operational bottleneck. The absence of automated AI-based filtering for copyright or inappropriate content represents a scalability challenge that would require third-party computer vision integration in a live environment.

Chapter 08: Discussion and Summary

8.1. Summary of the Report

This interim report has documented the systematic development of **Cre8tify**, from its inception, the identification of a gap in the custom apparel market in Sri Lanka to the creation of a functional **MERN** stack prototype. The research has established an effective architectural project foundation which includes a secure **JWT-based** authentication system, a role-based access control **(RBAC)** model for Designers and Buyers, and a scalable **NoSQL** database schema. Through the use of rigorous API testing, implementation has confirmed that the core backend logic is technically ready to support the ultimate integration of the interactive Design Toolbox.

8.2. Deviations from the Original Proposal

Since the Project Initiation Document (PID), two strategic pivots were made to enhance system quality:

- **Technological Shift:** The project transitioned from vanilla JavaScript to **TypeScript**. This change was necessitated by the increasing complexity of the customization logic, where static typing was required to ensure the maintainability of the frontend state.

- **Workflow Adjustment:** The original plan for a singular Stripe gateway was expanded into a **Hybrid Payment Model**. This integration utilize a **Sandbox environment** for simulated digital transactions alongside a **Manual Bank-Slip Verification** module. This dual-path strategy addresses the specific accessibility needs of the Sri Lankan market while maintaining the technical framework for automated processing.

- **Data Architecture:** The "Design" schema was further normalized to separate layers from the primary document, improving query performance for the marketplace grid.

8.3. Discussion of Implementation and Results

The implementation phase confirmed the viability of using a **NoSQL (MongoDB)** approach for design-heavy platforms. With the flexibility of the document model, design metadata was easily added without the need for schema migrations. The use of React.js to render the UI on the client side has also achieved the near-instant user interface (UI) responsiveness necessary for a creative platform. The initial challenges presented by CORS and Multer file-handling have been resolved with solutions like synchronous file unlinking, which are now keeping the server secure and optimized. As of this point in time, the system is already demonstrating stability, security, and readiness for the development of advanced features.

8.4. Future Plans and Upcoming Work

The project is currently on schedule, transitioning from the "Infrastructure" phase to the "Creative Engine" phase. The final development cycle will focus on:

- **Interactive Design Engine:** Integrating **Fabric.js** to develop the drag-and-drop design toolbox for real-time t-shirt customization.
- **Search and Discovery:** Implementing a categorized filtering system to allow Buyers to navigate the marketplace efficiently.
- **Quality Assurance:** Executing comprehensive Unit Testing on all API endpoints and performing User Acceptance Testing (UAT) with a cohort of local designers.
- **Cloud Deployment:** Migrating the local development environment to a production-ready cloud hosting service (e.g. AWS or Vercel) for public access.

References

Auth0 (2026). *JSON Web Token (JWT) Introduction*. [online] Available at: https://jwt.io/introduction/ (Accessed: 3 March 2026).

ECDB (2024). *Fashion - Sri Lanka | eCommerce market analysis*. [online] Available at: https://ecommercedb.com/markets/lk/fashion (Accessed: 14 January 2026).

Express.js (2025). *Express - Fast, unopinionated, minimalist web framework for Node.js*. [online] Available at: https://expressjs.com/ (Accessed: 3 March 2026).

Facebook Open Source (2025). *React: A JavaScript library for building user interfaces*. [online] Available at: https://reactjs.org/ (Accessed: 14 January 2026).

Fernando, P., Silva, R. and Kumara, A. (2024). 'Digital entrepreneurship in emerging economies: navigating payment and technical barriers in Sri Lanka', *Journal of Global Business Insights*, 9(1), pp. 45-60.

Google Developers (2025). *Web Vitals: Essential metrics for a healthy site*. [online] Available at: https://web.dev/vitals/ (Accessed: 14 January 2026).

Hasan, R. and Nguyen, B. (2025). 'Consumer trust and perceived risk in virtual fitting rooms', *Journal of Retailing and Consumer Services*, 78, p. 103722. doi:10.1016/j.jretconser.2024.103722.

Liu, J. and Wang, Y. (2024). 'MYCloth: Intelligent interaction and 3D visualization in apparel e-commerce', *International Journal of Fashion Design, Technology and Education*, 17(3), pp. 312-325.

MDN Web Docs (2024). *mix-blend-mode - CSS: Cascading Style Sheets*. [online] Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/mix-blend-mode (Accessed: 14 January 2026).

Meta Platforms, Inc. (2026). *React – A JavaScript library for building user interfaces*. [online] Available at: https://react.dev/ (Accessed: 14 January 2026).

Microsoft Corporation (2025). *TypeScript Documentation: Typed JavaScript at Any Scale*. [online] Available at: https://www.typescriptlang.org/ (Accessed: 14 January 2026).

MongoDB, Inc. (2026). *The MongoDB 7.0 Manual: Core Concepts and Architecture*. [online] Available at: https://www.mongodb.com/docs/manual/ (Accessed: 3 March 2026).

Mongoose (2026). *Mongoose v8.0.0: Elegant MongoDB object modeling for Node.js*. [online] Available at: https://mongoosejs.com/docs/ (Accessed: 3 March 2026).

Multer (2025). *Multer: Node.js middleware for handling multipart/form-data*. [online] Available at: https://github.com/expressjs/multer (Accessed: 3 March 2026).

Node.js Foundation (2025). *Node.js v20.x Documentation*. [online] Available at: https://nodejs.org/docs/ (Accessed: 14 January 2026).

NPM (2025). *Bcrypt.js: Optimized bcrypt in JavaScript with zero dependencies*. [online] Available at: https://www.npmjs.com/package/bcryptjs (Accessed: 3 March 2026).

Özbek, A. (2020). 'Online customized T-shirt design and evaluation of online websites for customization', *Industria Textila*, 71(4), pp. 371–379. [online] Available at: https://www.revistaindustriatextila.ro/ (Accessed: 14 January 2026).

Photopea (2026). *Photopea Online Photo Editor*. [online] Available at: https://www.photopea.com/ (Accessed: 14 January 2026).

Postman (2026). *Postman API Platform: API Tools for Developers*. [online] Available at: https://www.postman.com/ (Accessed: 3 March 2026).

Printify (2026). *Printify: Print on Demand & Drop Shipping Platform*. [online] Available at: https://printify.com/ (Accessed: 3 March 2026).

Redbubble (2026). *Redbubble: Awesome products from independent artists*. [online] Available at: https://www.redbubble.com/ (Accessed: 3 March 2026).

Siriwimala, M. and Liyanage, S. (2021). 'Technical barriers to e-commerce adoption among Sri Lankan creative professionals', *Sri Lanka Journal of Technology*, 12(4), pp. 201-215. [online] Available at: https://www.sljt.lk/ (Accessed: 14 January 2026).

Small and Medium Enterprises Sri Lanka (2025). *Digital adoption and payment infrastructure barriers for SMEs*. [online] Available at: https://www.srilankasme.lk/ (Accessed: 14 January 2026).

Technavio (2025). *Custom Apparel Market - Global Forecast and Analysis 2025-2029*. [online] Available at: https://www.technavio.com/ (Accessed: 14 January 2026).

Teespring (2026). *Teespring: Free platform to create and sell*. [online] Available at: https://teespring.com/ (Accessed: 3 March 2026).

University of Manchester (2024). 'Gen Z and the rise of Consumer Co-creation in digital fashion', *Creative Industries Research Hub*. [online] Available at: https://www.manchester.ac.uk/ (Accessed: 14 January 2026).

W3Schools (2025). *CSS Filters Reference*. [online] Available at: https://www.w3schools.com/cssref/css3_pr_filter.php (Accessed: 14 January 2026).

Zhang, T., Lu, C. and Kizildag, M. (2019). 'Banking on "What's Hot": Determinants of consumers' use of fashion apps', *Journal of Consumer Marketing*, 36(6), pp. 751-761.